ChatGPT 说：

你的电脑可以完成 P10 的模型实现和小规模训练全流程；课程原配置的完整预训练，以及真正的多卡 DDP 实验，需要服务器。

按你此前提供的配置：**Windows 11、华硕笔记本、RTX 4060 Laptop，8GB 显存**。NVIDIA 官方规格也确认这款 GPU 的显存为 8GB。下面的判断主要基于显卡；你的内存容量和剩余磁盘空间，我目前还不知道。 NVIDIA

复现目标	你的电脑能否完成
手写完整 GPT-2 124M 结构	**可以**，无需先缩小模型
加载 GPT-2 预训练权重、生成文本	**可以**
从随机初始化开始训练、观察 loss 下降	**可以**，需要减小 batch
实验混合精度、SDPA、`torch.compile`	**可以开展**，编译部分需合适的软件环境
梯度累积、验证集评估、HellaSwag 评估	**可以**，调整运行规模和频率
实际验证多 GPU DDP	**单张显卡无法完成**
按课程规模训练约 10B tokens	**单卡原则上能训练，但耗时较长，不建议作为本地学习的必做项**

我建议你：**本机保留完整的 124M 模型，按课程顺序完成各模块；最后再决定是否迁移到服务器做完整预训练。**

下面按原视频的时间轴安排。时间戳对应英文原版，部分转载版本可能略有偏移。

先准备课程资料和环境

使用这两个入口：

- P10 原视频：Let's reproduce GPT-2 (124M)
- 课程仓库：karpathy/build-nanogpt

优先跟 `build-nanogpt`，它的提交历史就是配合这节课逐步搭建的。你可以自己从空文件开始写，每完成一阶段，再对照对应提交。当前仓库还包含视频发布后的勘误。 GitHub

建议单独创建环境，避免影响你之前的学习项目：

Bash

```
conda create -n karpathy-p10 python=3.11 -y
conda activate karpathy-p10
```

先按 PyTorch 官方安装页面 选择适合驱动的 CUDA 版本安装 PyTorch，再安装：

Bash

```
python -m pip install numpy tiktoken transformers datasets tqdm requests matplotlib
```

验证当前环境：

Python

```
import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    p = torch.cuda.get_device_properties(0)
    print("GPU:", p.name)
    print("VRAM:", round(p.total_memory / 1024**3, 2), "GiB")
    print("BF16 supported:", torch.cuda.is_bf16_supported())
```

前半段可以先在 Windows 中做；为了贴近后面的编译和 Linux 训练环境，我更建议使用 WSL2 + Ubuntu。微软官方支持在 WSL2 中使用 NVIDIA CUDA 和 PyTorch。无需为了开始 P10，先折腾双系统。 Microsoft Learn

---

阶段 1：实现 GPT-2，并加载预训练权重｜00:03:39—00:33:31

这一阶段先完成“我写的模型，能否正确执行 GPT-2 的计算”。

按视频顺序实现：

1. `GPTConfig`
2. `CausalSelfAttention`
3. `MLP`
4. `Block`
5. `GPT`
6. `from_pretrained()`

保留 GPT-2 small 的配置：

Python

```
n_layer = 12
n_head = 12
n_embd = 768
block_size = 1024
vocab_size = 50257
```

本机先用很小的输入，例如 `B=1、T=16`，观察：

纯文本

```
输入 token IDs:   (B, T)
嵌入表示:         (B, T, 768)
输出 logits:      (B, T, 50257)
```

你需要重点理解：

- Q、K、V 如何拆分成多个注意力头。
- 因果 mask 为什么阻止当前位置看到未来。
- 残差连接和 LayerNorm 在哪里。
- token embedding 与 position embedding 如何相加。

加载 Hugging Face 权重时，特别留意部分矩阵需要转置：课程使用 `nn.Linear`，原权重中的一些层使用另一种权重布局。 GitHub

验收：所有权重成功加载；同一输入下，自己的模型与 Hugging Face 模型在 `eval()`、FP32 条件下输出接近。比较输出比“代码没报错”更有说服力。

阶段 2：实现文本生成｜00:33:31—00:45:50

按顺序完成：

1. 用 `tiktoken` 的 GPT-2 tokenizer 编码提示词。
2. 前向传播。
3. 取最后一个位置的 logits。
4. softmax 得到概率。
5. top-k 采样。
6. 将新 token 拼到输入后，继续生成。

这里正好承接你正在学习的 P9：**先使用现成的 GPT-2 tokenizer，保持分词结果与预训练权重一致。**

建议从一条提示词、生成 30—50 个 token 开始。理解核心关系：

$$P(x_{t+1} \mid x_1, \ldots, x_t) = \text{softmax}(\text{logits}_t)$$

验收：加载预训练权重后，能生成连贯的英文续写。这个阶段你的 4060 足够。

阶段 3：数据、交叉熵与单批次过拟合｜00:45:50—01:06:14

从这里开始，切换为**随机初始化模型**，学习训练流程。

先使用仓库的 `input.txt`，完成：

- 文本编码和连续 token 流。
- 输入、目标错开一个位置。
- 交叉熵损失。
- `zero_grad → forward → backward → step`。
- 简单的 `DataLoaderLite`。

例如：

```python
x = tokens[:B*T].view(B, T)
y = tokens[1:B*T+1].view(B, T)
```

你的起步配置：

```python
B = 1
T = 128
```

先固定同一批数据，训练几十到几百步，确认 loss 显著下降，再接入持续提供新数据的 loader。

初始化时，如果预测接近均匀分布，交叉熵大约为：

$$L \approx \ln(50257) \approx 10.82$$

这是检查损失量级的参考，实际初始值不必恰好等于它。

验收：固定 batch 的 loss 能下降；你能解释 `(B,T,V)` 的 logits 如何与 `(B,T)` 的目标计算损失。

阶段 4：权重共享与初始化｜01:06:14—01:22:18

跟着视频加入：

- token embedding 与输出层的权重共享。
- 线性层、嵌入层的初始化。
- 残差分支输出投影的初始化缩放。

权重共享应当让两个模块引用**同一个参数对象**，例如：

```python
model.lm_head.weight = model.transformer.wte.weight
```

可以检查：

```python
assert (
    model.lm_head.weight.data_ptr()
    == model.transformer.wte.weight.data_ptr()
)
```

验收：共享后，去重统计的参数量约为 124M；重新运行上一阶段，训练仍然正常。

对你来说，这一步值得认真看：你已经做过 makemore 的初始化和反向传播练习，现在可以观察这些问题如何出现在深层 Transformer 中。

阶段 5：逐项优化训练速度｜01:22:18—02:14:55

按课程顺序实验，不要一次全部打开：

优化项	你需要观察什么
TF32	FP32 矩阵乘法的速度与数值变化

优化项	你需要观察什么
BF16 混合精度	显存占用、吞吐量、loss 是否正常
`torch.compile`	首次编译开销与稳定运行后的速度
SDPA / FlashAttention	注意力计算的显存和速度变化
词表补齐到 50304	矩阵形状变化带来的性能影响

课程使用的注意力接口是：

Python

```
y = torch.nn.functional.scaled_dot_product_attention(
    q, k, v,
    is_causal=True,
)
```

这里先用 PyTorch 内置接口即可。SDPA 会根据输入和运行环境选择实现；调用这个接口本身，并不保证每次都使用 FlashAttention 内核。 PyTorch 2.15 documentation

你的实验方式：

- 比较优化效果时，保持模型、`B`、`T` 一致。
- 排除首次编译和预热步骤，再统计耗时。
- 每次记录 `tokens/s`、峰值显存和 loss。
- 优化稳定后，再尝试增大输入规模。

推荐逐渐尝试：

纯文本

```
B=1, T=128
    ↓
B=1, T=256
    ↓
B=1, T=1024，配合 BF16 + SDPA
    ↓
显存有余量，再尝试 B=2
```

这些是**试运行起点**，实际能放多大仍需测量。

为什么不能直接抄课程后期的 `B=64、T=1024`？仅一个 BF16 logits 张量就约占：

$$64 \times 1024 \times 50304 \times 2 \approx 6.6\ \text{GB}$$

再加上约 2GB 的 FP32 参数、梯度和 Adam 状态，以及各层激活，已经超过你的显存容量。

阶段 6：优化器、学习率与梯度累积｜02:14:55—02:46:52

这一阶段加入：

- AdamW 与参数分组。
- 梯度裁剪。
- 学习率 warmup。
- cosine decay。
- 梯度累积。

重点掌握这个公式：

$$\boxed{\text{每次更新的 token 数} = B \times T \times G \times W}$$

其中：

- B：每张卡一次前向传播的序列数。
- T：序列长度。
- G：梯度累积次数。
- W：GPU 数量。

例如，你用：

```python
B = 1
T = 1024
G = 16
W = 1
```

那么一次参数更新使用：

$$1 \times 1024 \times 16 \times 1 = 16384\ \text{tokens}$$

累积时，每个 micro-batch 都要及时反向传播：

```python
```

```
optimizer.zero_grad()

for _ in range(grad_accum_steps):
    # 获取下一批数据
    with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
        _, loss = model(x, y)

    (loss / grad_accum_steps).backward()

torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

不要把多个 micro-batch 的计算图全部留到最后再反向传播，否则会失去梯度累积节省显存的作用。

课程目标是每次更新约 524288 tokens。单卡 `B=1、T=1024` 时，需要累积 **512 次**。这可以保留更新批量，但每次更新要做更多轮串行计算。 GitHub

验收：你能算清“micro-step”“optimizer step”和“训练 token 数”之间的关系，且学习率按参数更新次数推进。

阶段 7：DDP 多卡训练｜02:46:52—03:10:21

这一阶段你的本机适合完成：

- 编写并理解 `rank、local_rank、world_size`。
- 理解每张卡如何读取不同的数据。
- 理解梯度同步和主进程日志。
- 保留普通单卡运行分支。

真正的多卡同步、通信开销和加速效果，需要至少两张 GPU 才能验证。单卡启动一个分布式进程，只能检查部分启动逻辑。

如果你之后仍能使用此前的双 3090 服务器，可以把这一阶段迁过去做。在此之前，继续用单卡分支完成后续的数据与评估模块。

阶段 8：FineWeb-Edu、验证集和 HellaSwag｜03:10:21—03:43:05

按课程顺序完成三个模块。

① 数据预处理

学习 `fineweb.py` 如何：

- 读取 FineWeb-Edu。
- 使用 GPT-2 tokenizer 编码。
- 在文档之间加入结束标记。
- 将 token 保存为 `uint16` 分片。
- 为训练和验证准备数据。课程脚本使用 `sample-10BT` 子集。 raw.githubusercontent.com

本地学习建议先处理 **1000万—5000万 tokens**。为避免下载完整数据后才截取，可以使用流式读取并限制处理量。

仅 10B 个 `uint16` token 的输出就约为 **20GB**，原始数据缓存还会额外占空间。

② 验证损失

加入 `model.eval()` 和 `torch.no_grad()`，使用固定验证数据，定期记录验证 loss。这样才能区分“训练集记住了”和“对未见数据有所改善”。

③ HellaSwag

先用少量样例跑通，理解如何根据候选续写的损失选择答案，再做完整评估。评分要处理好候选答案区域的 mask 和长度归一化。 GitHub

本地调试时降低评估规模、减少评估频率；完整评估留给阶段性检查点，避免大部分时间都花在评估上。

验收：同一个 checkpoint 能完成验证、文本生成和 HellaSwag 评分。

阶段 9：完整训练与结果分析｜03:43:05—结束

到这里，你已经可以完成一个端到端实验。建议先运行下面这套**本地学习配置**：

配置项	本地起步建议	课程最终代码的主要配置
模型	GPT-2 124M	GPT-2 124M
序列长度 `T`	256	1024
每卡 batch `B`	1	64
每次更新 token 数	4096	524288
梯度累积次数	单卡时为 16	根据 GPU 数量计算
参数更新次数	1000	19073

配置项	本地起步建议	课程最终代码的主要配置
warmup 次数	100	715
最大学习率	可先试 `3e-4`	`6e-4`

右列对应课程仓库中的训练设定；约 19073 次更新对应约 10B tokens。**左列是我为你的电脑安排的教学实验，会改变训练条件，不能直接期待课程中的最终指标。** GitHub

本地实验要留下这些结果：

- 训练和验证 loss 曲线。
- 固定提示词在不同 checkpoint 下的生成结果。
- 吞吐量和显存记录。
- 保存并重新加载模型后的评估结果。

如果准备长时间训练，还应保存优化器、训练步数和数据读取位置，保证中断后能够继续。

完整预训练需要多久，应根据你实测的**训练吞吐量**估算：

$$\text{训练时间} \approx \frac{10^{10}}{\text{训练 tokens/s}}$$

以下仅是换算示例，**不是对你电脑的测速结果**：

实测训练吞吐量	处理 10B tokens 所需时间
5000 tokens/s	约 23.1 天
10000 tokens/s	约 11.6 天
20000 tokens/s	约 5.8 天

这些时间还没有包含评估、保存和其他开销。因此，你可以先完成约 1000 步的本地实验，再用实际速度决定是否值得继续，或迁移到服务器。

你现在可以直接从阶段 1 开始，保留完整的 12 层、12 头、768 维模型；先把输入缩小。这样既能贴着课程学习，也能让之后迁移到服务器时复用同一套模型和训练代码。